



Annexe — Spring Boot Crash Course

Année 2026–2027 Enseignant Bachar Rima

À QUOI SERT CE DOCUMENT

Donner le modèle mental minimal pour **comprendre ce que fait Spring Boot** : pourquoi un framework, ce qu'est l'inversion de contrôle, ce que gère le conteneur, comment une application Spring Boot démarre et s'arrête. Il sert à la **partie B facultative du TP 1**, puis aux TP suivants de l'UE, où Spring Boot est utilisé de façon systématique. Ce n'est pas un manuel de Spring : seul ce qui sert dans l'UE est présenté.

ÉLÉMENT	VERSION	REMARQUE
JDK	17 minimum	exigé par Spring Boot 3 ; exemples validés sur 17.0.10 et 21.0.5
Spring Boot	3.5.16	version épinglée dans tous les <code>pom.xml</code>
Maven	3.6.3 minimum	exigence de Spring Boot 3.5 ; validé avec 3.9.6
Dépendance	<code>spring-boot-starter</code>	aucune dépendance web

EXEMPLES EXÉCUTABLES

Tous les exemples de ce document sont dans [resources/spring-boot-crash-course/](#) :

- `etape1-sans-spring` — une petite application assemblée *à la main*, en Java seul ;
- `etape2-spring-boot` — *la même* application, assemblée par Spring Boot.

Le domaine concerne une médiathèque qui enregistre des emprunts.

1. Pourquoi un framework ?

Une **bibliothèque** est un ensemble de classes que *votre code* appelle : vous décidez quand et comment. Un **framework** (canevas) est une application partielle qui *appelle votre code* aux endroits qu'il a prévus : vous fournissez les pièces, il décide de l'enchaînement.



FIGURE 1 – Qui appelle qui : bibliothèque et framework

Ce renversement porte un nom : **inversion de contrôle** (IoC), parfois résumée par le « principe d'Hollywood » — *ne nous appelez pas, c'est nous qui vous appellerons*. Le CM 01 (section 4) le présente comme le trait distinctif d'un framework.

Le bénéfice est concret : le framework prend en charge ce qui est *répétitif et difficile à bien faire* — démarrage, assemblage des objets, configuration, arrêt propre, journalisation — et vous laisse le métier.

2. Couplage et dépendances

Une classe **A** **dépend** de **B** si elle ne peut pas fonctionner sans elle. Le **couplage** mesure à quel point un changement de **B** oblige à changer **A**. Un couplage fort rend le code difficile à tester, à réutiliser et à faire évoluer.

Deux principes, parmi les cinq regroupés sous l'acronyme SOLID, suffisent à comprendre ce que fait Spring :

PRINCIPE	CE QU'IL DEMANDE
Responsabilité unique (S)	une classe n'a qu'une raison de changer : le service d'emprunt ne sait pas comment on stocke un livre ni comment on envoie une notification
Inversion des dépendances (D)	le code de haut niveau dépend d' <i>abstractions</i> (des interfaces), pas de classes concrètes : le service dépend de <code>Catalogue</code> , pas de <code>CatalogueEnMemoire</code>

LES TROIS AUTRES PRINCIPES SOLID

Ouvert/fermé (étendre sans modifier), substitution de Liskov (un sous-type doit pouvoir remplacer son type), ségrégation des interfaces (préférer plusieurs petites interfaces). Utiles en conception, ils ne sont pas nécessaires pour comprendre le conteneur.

3. L'injection de dépendances, sans framework

Appliquer l'inversion des dépendances pose une question : si le service ne crée pas ses collaborateurs, *qui* les crée ? Réponse : quelqu'un d'autre les crée et les lui **injecte**, le plus souvent par son **constructeur**. C'est l'**injection de dépendances**. Elle ne nécessite aucun framework :

Listing 1 – étape 1 — Catalogue.java

```
1 package mediatheque;
2
3 import java.util.Optional;
4
5 /** Abstraction du stockage : le service n'en connaît pas la réalisation. */
6 public interface Catalogue {
7     Optional<Livre> trouver(String isbn);
8 }
```

Listing 2 – étape 1 — EmpruntService.java

```
1 package mediatheque;
2
3 import java.time.Clock;
4 import java.time.LocalDate;
5
6 /**
7  * Le service métier. Il DEPEND de trois collaborateurs, mais ne les
8  * crée pas : ils lui sont fournis par son constructeur.
9  * C'est déjà de l'injection de dépendances - sans aucun framework.
10  */
11 public class EmpruntService {
12
13     private final Catalogue catalogue;
14     private final Notificateur notificateur;
15     private final Clock horloge;
16     private final int dureeJours;
```

```

17
18     public EmpruntService(Catalogue catalogue, Notificateur notificateur,
19                           Clock horloge, int dureeJours) {
20         this.catalogue = catalogue;
21         this.notificateur = notificateur;
22         this.horloge = horloge;
23         this.dureeJours = dureeJours;
24     }
25
26     public String emprunter(String isbn, String lecteur) {
27         Livre livre = catalogue.trouver(isbn)
28             .orElseThrow(() ->
29                 new IllegalArgumentException("isbn inconnu : " + isbn));
30         LocalDate retour = LocalDate.now(horloge).plusDays(dureeJours);
31         notificateur.notifier(lecteur + " emprunte \"" + livre.titre() + "\"");
32         return "retour attendu le " + retour;
33     }
34 }

```

Le service reçoit trois collaborateurs et une valeur de configuration. Il ne sait pas quelles classes concrètes il utilise. Quelqu'un doit pourtant les choisir et les assembler : c'est la **racine de composition**.

Listing 3 – étape 1 — Main.java

```

1  package mediatheque;
2
3  import java.time.Clock;
4  import java.time.Instant;
5  import java.time.ZoneOffset;
6
7  /**
8   * La "RACINE DE COMPOSITION" : le seul endroit qui connaît les classes
9   * concrètes et les assemble. Ici, c'est nous qui le faisons à la main.
10  */
11  public class Main {
12
13      public static void main(String[] args) {
14          // horloge fixe : sortie reproductible
15          Clock horloge =
16              Clock.fixed(Instant.parse("2026-09-14T09:00:00Z"), ZoneOffset.UTC);
17
18          Catalogue catalogue = new CatalogueEnMemoire();
19          Notificateur notificateur = new NotificateurConsole();
20          EmpruntService service = new EmpruntService(catalogue, notificateur, horloge, 21);
21
22          System.out.println(service.emprunter("978-2", "alice"));
23      }
24  }

```

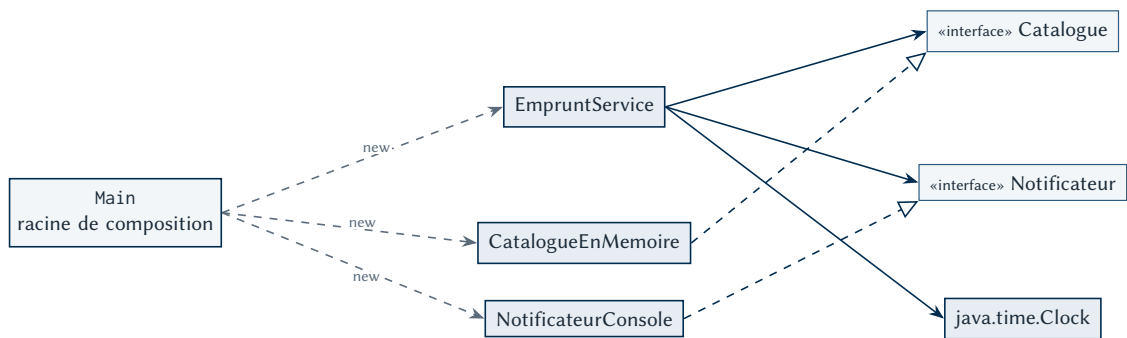


FIGURE 2 – Étape 1 : la racine de composition assemble tout à la main

C'est propre, testable, et parfaitement suffisant pour une petite application. Mais quand les classes se comptent par dizaines, la racine de composition devient longue, fragile et répétitive ; il faut aussi gérer la configuration, l'ordre de démarrage et l'arrêt. C'est ce travail qu'un **conteneur IoC** automatise.

4. Le conteneur IoC

Un conteneur d'inversion de contrôle est un objet qui **crée, assemble et gère le cycle de vie** des objets d'une application à partir d'une description. Dans Spring, ces objets s'appellent des **beans**, et le conteneur est l'**ApplicationContext**.

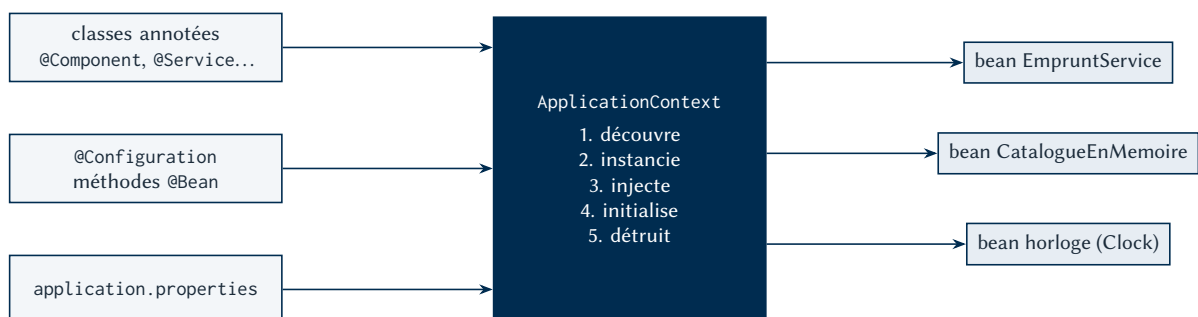


FIGURE 3 – Le conteneur construit la racine de composition à votre place

NOTION	DÉFINITION
Bean	un objet <i>créé et géré</i> par le conteneur. Un objet créé par <code>new</code> dans votre code n'en est pas un, même si sa classe porte <code>@Service</code> .
Définition de bean	ce que le conteneur sait de lui : classe, dépendances, portée, méthodes de cycle de vie.
ApplicationContext	le conteneur lui-même : il construit les beans, les injecte, publie des événements, charge la configuration.
Portée (scope)	combien d'instances. Par défaut singleton : une seule instance par contexte, partagée. Prototype : une nouvelle instance à chaque demande. Les portées <i>request</i> et <i>session</i> n'existent que dans une application web.

5. Spring Framework et Spring Boot

	SPRING FRAMEWORK	SPRING BOOT
Rôle	le conteneur IoC et de nombreux modules (accès aux données, web, messagerie, sécurité...)	une façon <i>de démarrer</i> des applications Spring avec un minimum de configuration
Apporte	<code>ApplicationContext</code> , beans, injection, AOP	démarrage par <code>SpringApplication</code> , <i>starters</i> , auto-configuration, configuration externalisée, JAR exécutable
Version (2026–2027)	6.2.x	3.5.16 (fondé sur Spring Framework 6.2)

Deux mécanismes font la réputation de Spring Boot :

- **les *starters*** : une dépendance Maven qui en regroupe d'autres, cohérentes et de versions compatibles. `spring-boot-starter` apporte le conteneur, la configuration et la journalisation ; `spring-boot-starter-web` y ajouterait un serveur HTTP. Les versions viennent du *parent* `spring-boot-starter-parent` : on ne les écrit pas.
- **l'*auto-configuration*** : au démarrage, Spring Boot regarde ce qui est présent sur le *classpath* et crée les beans d'infrastructure correspondants. Avec le seul starter de base, il n'y a rien de web à configurer : l'application n'est pas une application web.

6. La même application, assemblée par Spring Boot

L'étape 2 reprend **exactement** les classes de l'étape 1. `Livre`, `Catalogue` et `Notificateur` sont identiques. Les autres reçoivent quelques annotations, et `Main` disparaît.

6.1. Le scan des composants et les stéréotypes

Une classe annotée par un **stéréotype** est détectée par le *scan des composants* et devient un bean.

Listing 4 – étape 2 — `NotificateurConsole.java`

```
1 package mediatheque;
2
3 import org.springframework.stereotype.Component;
4
5 /** @Component : un bean generique, detecte par le scan des composants. */
6 @Component
7 public class NotificateurConsole implements Notificateur {
8
9     @Override
10    public void notifier(String message) {
11        System.out.println("[notification] " + message);
12    }
13 }
```

Listing 5 – étape 2 — `CatalogueEnMemoire.java`

```
1 package mediatheque;
2
3 import java.util.HashMap;
4 import java.util.Map;
5 import java.util.Optional;
6
7 import jakarta.annotation.PostConstruct;
```

```

8 import jakarta.annotation.PreDestroy;
9 import org.springframework.stereotype.Repository;
10
11 /**
12  * @Repository : un bean dont le role est l'accès aux données.
13  * @PostConstruct / @PreDestroy : points d'accroche du cycle de vie.
14  */
15 @Repository
16 public class CatalogueEnMemoire implements Catalogue {
17
18     private final Map<String, Livre> livres = new HashMap<>();
19
20     @PostConstruct
21     void charger() {
22         livres.put("978-2", new Livre("978-2", "Les Miserables"));
23         livres.put("978-3", new Livre("978-3", "Germinal"));
24         System.out.println("[cycle de vie] catalogue charge : "
25             + livres.size() + " livres");
26     }
27
28     @PreDestroy
29     void fermer() {
30         System.out.println("[cycle de vie] catalogue ferme");
31     }
32
33     @Override
34     public Optional<Livre> trouver(String isbn) {
35         return Optional.ofNullable(livres.get(isbn));
36     }
37 }

```

ANNOTATION	USAGE
@Component	stéréotype générique : « ceci est un bean »
@Service	un bean de logique métier ; même effet technique, intention plus claire
@Repository	un bean d'accès aux données ; Spring peut en plus y traduire les exceptions de persistance
@Controller, @RestController	beans web : <i>hors sujet</i> pour une application non web, vus avec REST

Le scan part du packaging de la classe annotée `@SpringBootApplication` et descend dans ses sous-packetages. **Une classe placée ailleurs n'est pas trouvée** — c'est l'une des erreurs les plus fréquentes (§ 10).

6.2. L'injection par constructeur

Listing 6 – étape 2 — `EmpruntService.java`

```

1 package mediatheque;
2
3 import java.time.Clock;
4 import java.time.LocalDate;
5
6 import org.springframework.beans.factory.annotation.Value;
7 import org.springframework.stereotype.Service;
8
9 /**
10  * Le MEME service qu'a l'etape 1. Deux ajouts seulement :

```

```

11 * - @Service, pour que le conteneur le cree ;
12 * - @Value, pour lire la duree dans application.properties.
13 * Un seul constructeur : Spring l'utilise, @Autowired est inutile.
14 */
15 @Service
16 public class EmpruntService {
17
18     private final Catalogue catalogue;
19     private final Notificateur notificateur;
20     private final Clock horloge;
21     private final int dureeJours;
22
23     public EmpruntService(Catalogue catalogue, Notificateur notificateur, Clock horloge,
24         @Value("${mediatheque.duree-emprunt-jours}") int dureeJours) {
25         this.catalogue = catalogue;
26         this.notificateur = notificateur;
27         this.horloge = horloge;
28         this.dureeJours = dureeJours;
29     }
30
31     public String emprunter(String isbn, String lecteur) {
32         Livre livre = catalogue.trouver(isbn)
33             .orElseThrow(() ->
34                 new IllegalArgumentException("isbn inconnu : " + isbn));
35         LocalDate retour = LocalDate.now(horloge).plusDays(dureeJours);
36         notificateur.notifier(lecteur + " emprunte \"" + livre.titre() + "\"");
37         return "retour attendu le " + retour;
38     }
39 }

```

Spring voit **un seul constructeur** et l'utilise : il cherche un bean pour chaque paramètre et les passe. Depuis Spring 4.3, `@Autowired` est inutile dans ce cas. `@Value` injecte une valeur lue dans la configuration.

PRÉFÉREZ LE CONSTRUCTEUR À L'INJECTION SUR ATTRIBUT

On rencontre souvent `@Autowired` posé directement sur un attribut. Cela fonctionne, mais :

- l'attribut ne peut pas être `final`;
- les dépendances ne sont pas visibles dans la signature de la classe;
- on ne peut pas créer l'objet correctement dans un test sans conteneur.

L'injection par constructeur n'a aucun de ces défauts : c'est la forme recommandée par la documentation de Spring, et celle utilisée dans l'UE.

6.3. @Configuration et @Bean

`EmpruntService` a besoin d'une `java.time.Clock`. On ne peut pas annoter une classe du JDK : on écrit donc la *méthode* qui fabrique l'objet, et Spring enregistre son résultat comme bean.

Listing 7 – étape 2 — HorlogeConfig.java

```

1 package mediatheque;
2
3 import java.time.Clock;
4 import java.time.Instant;
5 import java.time.ZoneOffset;
6
7 import org.springframework.context.annotation.Bean;
8 import org.springframework.context.annotation.Configuration;
9
10 /**
11  * @Configuration + @Bean : pour un objet que l'on ne peut pas annoter.

```

```

12 * java.time.Clock est une classe du JDK : impossible d'y ajouter
13 * @Component. On écrit donc la méthode qui la fabrique, et Spring
14 * enregistre son résultat comme bean.
15 */
16 @Configuration
17 public class HorlogeConfig {
18
19     @Bean
20     public Clock horloge() {
21         // horloge fixe : sortie reproductible (en production : Clock.systemDefaultZone())
22         return Clock.fixed(Instant.parse("2026-09-14T09:00:00Z"), ZoneOffset.UTC);
23     }
24 }

```

Règle pratique : **stéréotype** pour vos propres classes, **@Bean** pour les objets que vous ne pouvez pas annoter (bibliothèques, JDK) ou dont la construction demande de la logique — un registre RMI, par exemple.

6.4. Le point d'entrée

Listing 8 – étape 2 — MediathequeApplication.java

```

1 package mediatheque;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 /**
7  * @SpringBootApplication = @Configuration + @EnableAutoConfiguration
8  *                          + @ComponentScan (à partir du paquetage "mediatheque").
9  *
10 * Plus aucun "new" : la racine de composition est le conteneur.
11 */
12 @SpringBootApplication
13 public class MediathequeApplication {
14
15     public static void main(String[] args) {
16         SpringApplication.run(MediathequeApplication.class, args);
17         System.out.println("[main] run() a rendu la main : le contexte est prêt");
18     }
19 }

```

@SpringBootApplication réunit trois annotations : **@Configuration**, **@EnableAutoConfiguration** et **@ComponentScan**. **SpringApplication.run** crée l'**ApplicationContext**, exécute l'auto-configuration, construit et injecte tous les beans, lance les *runners*, puis rend la main.

6.5. La configuration externalisée

Listing 9 – étape 2 — application.properties

```

1 # Configuration externalisée : lue au démarrage, injectée avec @Value
2 mediatheque.duree-emprunt-jours=21
3
4 # Sortie console réduite pour la démonstration
5 spring.main.banner-mode=off
6 logging.level.root=WARN

```

Le même contenu peut s'écrire en YAML, dans `application.yml` :

Listing 10 – équivalent YAML

```
1 mediatheque:
2   duree-emprunt-jours: 21
3 spring:
4   main:
5     banner-mode: "off"
```

Les guillemets autour de "off" sont nécessaires : en YAML, un `off` nu est lu comme le booléen `false`.

Une propriété peut être redéfinie au lancement, sans recompiler ; la date de retour devient alors le 2026-09-21 :

```
$ java -jar target/mediatheque-1.0.0.jar --mediatheque.duree-emprunt-jours=7
```

Pour de nombreuses propriétés liées, on préfère `@ConfigurationProperties`, qui les regroupe dans une classe.

7. Le cycle de vie d'une application non web

Listing 11 – étape 2 — DemoRunner.java

```
1 package mediatheque;
2
3 import java.util.Arrays;
4
5 import org.springframework.boot.ApplicationArguments;
6 import org.springframework.boot.ApplicationRunner;
7 import org.springframework.context.ApplicationContext;
8 import org.springframework.stereotype.Component;
9
10 /**
11  * ApplicationRunner : code execute UNE FOIS, quand le contexte est pret.
12  * C'est le "main" d'une application Spring Boot non web.
13  */
14 @Component
15 public class DemoRunner implements ApplicationRunner {
16
17     private final EmpruntService service;
18     private final ApplicationContext contexte;
19
20     public DemoRunner(EmpruntService service, ApplicationContext contexte) {
21         this.service = service;
22         this.contexte = contexte;
23     }
24
25     @Override
26     public void run(ApplicationArguments args) {
27         // parmi les dizaines de beans du contexte, ceux que NOUS avons declares
28         String[] nosBeans = Arrays.stream(contexte.getBeanDefinitionNames())
29             .filter(nom -> {
30                 Class<?> type = contexte.getType(nom);
31                 return type != null && (type.getPackageName().equals("mediatheque")
32                     || type == java.time.Clock.class
33                     || nom.equals("horloge"));
34             })
35             .sorted()
36             .toArray(String[]::new);
37         System.out.println("[contexte] beans de l'application :");
38         for (String nom : nosBeans) {
39             System.out.println("    " + nom);
```

```

40     }
41
42     System.out.println(service.emprunter("978-2", "alice"));
43
44     EmpruntService encore = contexte.getBean(EmpruntService.class);
45     System.out.println("[contexte] meme instance (singleton) ? "
46         + (encore == service));
47
48     try {
49         service.emprunter("000-0", "bob");
50     } catch (IllegalArgumentException e) {
51         System.out.println("[erreur métier] " + e.getMessage());
52     }
53 }
54 }

```

Un **ApplicationRunner** (ou son équivalent **CommandLineRunner**, qui reçoit les arguments bruts) est un bean dont la méthode **run** est appelée *une fois*, quand le contexte est prêt. C'est le point d'entrée du scénario d'une application sans interface web.

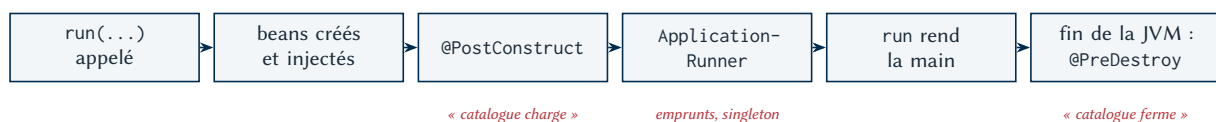


FIGURE 4 – Cycle de vie observé dans l'exemple

7.1. Exécution

```

$ cd etape2-spring-boot
$ mvn clean package
$ java -jar target/mediatheque-1.0.0.jar

```

```

[cycle de vie] catalogue charge : 2 livres
[contexte] beans de l'application :
    catalogueEnMemoire
    demoRunner
    empruntService
    horloge
    horlogeConfig
    mediathequeApplication
    notificateurConsole
[notification] alice emprunte "Les Miserables"
retour attendu le 2026-10-05
[contexte] meme instance (singleton) ? true
[erreur métier] isbn inconnu : 000-0
[main] run() a rendu la main : le contexte est pret
[cycle de vie] catalogue ferme

```

1. **[cycle de vie] catalogue charge** s'affiche *avant tout le reste* : le conteneur a créé le bean et appelé **@PostConstruct** pendant **run**.
2. La liste des beans contient **horloge** (la méthode **@Bean**), les classes annotées, et même **horlogeConfig** et **mediathequeApplication** : les classes de configuration sont elles-mêmes des beans.
3. **meme instance (singleton) ? true** : demander le bean au contexte rend l'objet déjà injecté dans le runner.
4. **run() a rendu la main** n'apparaît qu'*après* le runner.
5. **catalogue ferme** arrive en dernier : aucun fil d'exécution ne maintient la JVM, elle s'arrête, et le contexte est fermé proprement au passage — **@PreDestroy** est appelé.

RESTER EN VIE, OU S'ARRÊTER : TROIS SITUATIONS

- **Un traitement puis la fin** (l'exemple) : rien à faire, la JVM s'arrête quand les runners ont terminé.
- **Un service qui doit rester disponible** sans serveur web — par exemple un serveur RMI : ajouter `spring.main.keep-alive=true` (Spring Boot 3.2 et suivants), sans quoi l'application démarre puis s'arrête aussitôt.
- **Un programme qui a démarré des fils d'exécution qu'il ne gère pas lui-même** — un objet RMI exporté, par exemple — et qui doit s'arrêter à la fin de son scénario : fermer explicitement le contexte avec `System.exit(SpringApplication.exit(contexte))`, pour que les méthodes de destruction des beans libèrent ces ressources.

8. Le projet Maven

```
etape2-spring-boot/  
|-- pom.xml                                dependances, version de Spring Boot  
`-- src/  
    |-- main/  
    |   |-- java/  
    |   |   |-- mediatheque/                paquetage racine : point de depart du scan  
    |   |   |-- MediathequeApplication.java  
    |   |   |-- EmpruntService.java  
    |   |   |-- ...  
    |   |-- resources/  
    |       |-- application.properties      configuration externalisee
```

Listing 12 – étape 2 — pom.xml

```
1  <?xml version="1.0" encoding="UTF-8"?>  
2  <project xmlns="http://maven.apache.org/POM/4.0.0"  
3      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
4      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0  
5          https://maven.apache.org/xsd/maven-4.0.0.xsd">  
6      <modelVersion>4.0.0</modelVersion>  
7  
8      <parent>  
9          <groupId>org.springframework.boot</groupId>  
10         <artifactId>spring-boot-starter-parent</artifactId>  
11         <version>3.5.16</version>  
12         <relativePath/>  
13     </parent>  
14  
15     <groupId>fr.umontpellier.hai704i</groupId>  
16     <artifactId>mediatheque</artifactId>  
17     <version>1.0.0</version>  
18  
19     <properties>  
20         <java.version>17</java.version>  
21     </properties>  
22  
23     <dependencies>  
24         <!-- Le starter minimal : conteneur IoC, configuration, journalisation.  
25             Pas de serveur web. -->  
26         <dependency>  
27             <groupId>org.springframework.boot</groupId>  
28             <artifactId>spring-boot-starter</artifactId>  
29         </dependency>  
30     </dependencies>  
31  
32     <build>
```

```

33     <plugins>
34         <plugin>
35             <groupId>org.springframework.boot</groupId>
36             <artifactId>spring-boot-maven-plugin</artifactId>
37         </plugin>
38     </plugins>
39 </build>
40 </project>

```

- **Le parent** fixe la version de Spring Boot et, avec elle, celle de toutes les dépendances gérées : c'est la seule version à écrire.
- **Le starter** apporte le conteneur, la configuration et la journalisation.
- **Le plugin** `spring-boot-maven-plugin` produit un JAR exécutable par `java -jar`, et fournit `mvn spring-boot:run` pour lancer l'application sans emballer.

ÉPINGLER UNE VERSION

N'écrivez jamais « la dernière version » dans un projet qu'un autre doit reproduire. Écrivez une version précise, testez-la, et indiquez-la dans votre README.

9. Créer un projet avec Spring Initializr

start.spring.io engendre un squelette de projet ; la plupart des IDE Java proposent le même assistant ou savent importer le projet produit (*Import* → *Existing Maven Project*).

CHAMP	VALEUR POUR L'UE
Project	Maven
Language	Java
Spring Boot	la version proposée, puis corriger le <code>pom.xml</code> (voir ci-dessous)
Group / Artifact	par exemple <code>fr.umontpellier.hai704i</code> / <code>mediatheque</code>
Packaging	Jar
Java	17
Dependencies	aucune (ni Spring Web, ni JPA, ni base de données)

INITIALIZR NE PROPOSE QUE LES VERSIONS EN COURS DE SUPPORT

Au moment de la rédaction, start.spring.io ne propose plus la ligne 3.5 : il engendre des projets Spring Boot 4.x. Pour rester sur la version de référence de l'UE, remplacez simplement la version du parent dans le `pom.xml` engendré par `3.5.16` — ou partez du `pom.xml` ci-dessus. Les exemples de ce document ne dépendent d'aucune fonctionnalité propre à la version 4.

10. Erreurs fréquentes

Tous les messages ci-dessous ont été obtenus en provoquant l'erreur sur l'exemple de l'étape 2 (Spring Boot 3.5.16).

SYMPTÔME	CAUSE	CORRECTION
Parameter 0 of constructor in mediatheque.EmpruntService required a bean of type 'mediatheque.Catalogue' that could not be found.	aucune classe annotée ne réalise <code>Catalogue</code> — stéréotype oublié, ou classe hors du paquetage scanné	annoter la classe ; la placer sous le paquetage de <code>@SpringBootApplication</code>
required a single bean, but 2 were found	deux beans réalisent la même interface	n'en garder qu'un, ou désigner le bon avec <code>@Primary</code> ou <code>@Qualifier</code>
Tomcat started on port 8080 (http), et le programme ne s'arrête plus	<code>spring-boot-starter-web</code> ajouté par erreur	revenir à <code>spring-boot-starter</code>
le programme démarre puis s'arrête aussitôt, alors qu'il devait rester disponible	aucun fil d'exécution ne maintient la JVM	<code>spring.main.keep-alive=true</code>
le programme a fini son travail mais ne s'arrête pas	une ressource garde un fil actif (objet RMI exporté...)	fermer le contexte : <code>SpringApplication.exit</code>
release version 17 not supported à la compilation	Maven utilise un JDK antérieur à 17	pointer <code>JAVA_HOME</code> sur un JDK 17 ou plus
UnsupportedClassVersionError ... class file version 61.0	le JAR est lancé avec un JDK antérieur à 17	lancer avec <code>java 17</code> ou plus
<code>@PostConstruct</code> jamais appelé, dépendances manquantes	l'objet a été créé par <code>new</code> : ce n'est pas un bean	laisser le conteneur le créer, et le recevoir par injection

11. Spring Boot dans HAI704I

NOTION DE CE DOCUMENT	OÙ ELLE SERT
beans, stéréotypes, injection par constructeur	TP 1, partie B : le servant devient un <code>@Service</code> ; le stub est injecté côté client
<code>@Configuration</code> / <code>@Bean</code>	TP 1, partie B : le registre RMI, objet que l'on ne peut pas annoter
<code>ApplicationRunner</code> , <code>keep-alive</code> , fermeture du contexte	TP 1, partie B : publier au démarrage, rester disponible, s'arrêter proprement
configuration externalisée	tous les TP : ports, noms de services, adresses
starters et auto-configuration	TP suivants : les starters propres à chaque technologie

CE QUE SPRING BOOT NE CHANGE PAS

Spring Boot assemble et fait vivre votre application. Il ne change rien à la **communication distante** : un appel RMI reste un appel RMI, avec ses stubs, sa sérialisation et ses `RemoteException`. L'annexe « Java RMI avec Spring Boot » du TP 1 le montre sur le cabinet vétérinaire.

Récapitulatif des annotations utilisées dans l'UE

ANNOTATION	EFFET
@SpringBootApplication	configuration + auto-configuration + scan à partir de ce paquetage
@Component, @Service, @Repository	la classe devient un bean détecté par le scan
@Configuration	la classe déclare des beans par des méthodes @Bean
@Bean	le résultat de la méthode devient un bean
@Value("\${cle}")	injecte une valeur de configuration
@PostConstruct, @PreDestroy	méthodes appelées après l'injection, et à la fermeture du contexte (jakarta.annotation)
@Primary, @Qualifier	choisir parmi plusieurs beans du même type
@Autowired	inutile sur un constructeur unique ; déconseillée sur un attribut